

Security Misconfiguration — Interview Prep Notes

1. Logical Description

- Insecure security-relevant settings across any layer: app, framework, server, cloud, DB, network
- Root state: secure-by-default not enforced or overridden by convenience config
- Not a code vuln class — a **state/config** vuln class → detected via probing, not payload injection
- Umbrella term: catches what doesn't fit other named categories (headers, defaults, debug, verbose errors, unnecessary features/services, cloud storage, CORS wildcard subset, permissive CSP, exposed admin panels)

2. OWASP / CWE

- **OWASP 2021:** A05:2021-Security Misconfiguration
- **OWASP 2025 draft:** A02:2025 (renumbered, same concept)
- **OWASP API Top 10:** API8:2023-Security Misconfiguration
- **CWE-16:** Configuration (parent)
- **CWE-2:** Environmental Security Flaws
- **CWE-1004:** Sensitive Cookie w/o HttpOnly
- **CWE-548:** Directory Listing
- **CWE-209:** Info Exposure Through Error Msg
- **CWE-1032:** OWASP Top Ten 2017 Category A6 mapping (legacy ref, mention only if asked)
- **CWE-215:** Debug info exposure
- **CWE-732:** Incorrect Permission Assignment

3. Common Variants

- Default credentials (admin:admin, tomcat:tomcat, RabbitMQ guest:guest)
- Directory listing enabled
- Debug/dev mode in prod (Django DEBUG=True, Spring Boot Actuator, Flask debug console/Werkzeug)
- Verbose errors/stack traces (SQL errors, framework version leaks)
- Missing/misconfigured security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options)
- Exposed config/metadata files (.env, .git, web.config, .DS_Store, backup.zip)
- Unnecessary HTTP methods enabled (PUT, TRACE, DELETE)
- Cloud storage misconfig (public S3/GCS buckets, open blob storage)
- Overly permissive CORS as subset of misconfig

- Unpatched sample apps left on server (Tomcat manager, phpinfo.php)
- Excessive permissions on cloud IAM roles / service accounts
- Verbose HTTP response headers (Server, X-Powered-By)
- Improper CSP (unsafe-inline, wildcard sources)
- Unnecessary services running (FTP, Telnet, SSH weak ciphers)

4. Attack Surface / Entry Points

- ☐ HTTP response headers (security headers, Server/X-Powered-By)
- ☐ Admin/management consoles (Tomcat Manager, Jenkins, phpMyAdmin, Actuator)
- ☐ Cloud storage buckets/URLs
- ☐ VCS/metadata artifacts left in webroot (.git, .svn, .env, .DS_Store)
- ☐ Debug/profiling endpoints (/debug/pprof, /actuator/heapdump, Django debug toolbar)
- ☐ Error pages / stack traces (force via malformed input)
- ☐ Default install paths (/phpmyadmin, /manager/html, /solr)
- ☐ HTTP methods (OPTIONS, TRACE, PUT)
- ☐ Cloud metadata endpoints (169.254.169.254 — overlaps SSRF)
- ☐ API docs/spec left exposed (Swagger UI, /graphql introspection)
- ☐ Backup/log files in webroot

5. Testing Workflow

Step 1 — Recon/fingerprinting

- Verify: tech stack, versions, exposed panels
- Manual: `curl -I`, Wappalyzer, WhatWeb, `whatweb -a 3`
- Burp: Passive scan + Site Map, Wappalyzer extension
- ZAP: Passive scan, Tech Detection add-on
- Nuclei: `nuclei -u <target> -t technologies/`
- Payload rationale: none — passive fingerprint only

Step 2 — Header audit

- Verify: CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy present/correct
- Manual: `curl -I https://target`
- Burp: Response tab / HTTP History review
- ZAP: Passive rule "Missing security headers"
- Nuclei: `-t http/misconfiguration/http-missing-security-headers.yaml`
- Payload: none, header presence/value check

Step 3 — Sensitive file/path discovery

- Verify: .git, .env, backup files, config files reachable

- Manual: `curl -s target/.git/HEAD`, `ffuf -w wordlist -u target/FUZZ`
- Burp: Content Discovery / Intruder w/ wordlist
- ZAP: Forced Browse (DirBuster add-on)
- Nuclei: `-t exposures/` (existing templates cover .git, .env, backup)
- Payloads: `/.git/config`, `/.env`, `/web.config`, `/backup.zip`, `/.DS_Store`

Step 4 — Default credentials

- Verify: admin panels accept default/weak creds
- Manual: try known defaults (admin:admin, tomcat:tomcat, guest:guest)
- Burp: Intruder w/ creds wordlist against login
- ZAP: Fuzzer
- Nuclei: `-t default-logins/`
- Payload rationale: vendor-known default cred lists (SecLists Default-Credentials)

Step 5 — Debug/verbose error induction

- Verify: stack traces, debug consoles reachable
- Manual: malformed input (`'`, invalid type, huge payload) to trigger error; hit `/debug`, `/actuator`
- Burp: Repeater — send garbage params, inspect response
- ZAP: Active scan "Application Error Disclosure"
- Nuclei: `-t exposures/configs/springboot-heapdump.yaml`, `-t exposures/configs/django-debug.yaml`
- Payloads: `?debug=true`, `?XDEBUG_SESSION_START=1`, invalid JSON body, oversized int

Step 6 — HTTP method / verb tampering

- Verify: dangerous methods enabled
- Manual: `curl -X OPTIONS`, `curl -X TRACE`, `curl -X PUT -d @file`
- Burp: Repeater — change method
- ZAP: Active scan "HTTP methods"
- Nuclei: custom template only
- Payload rationale: confirm allowed methods vs actually enforced

Step 7 — Cloud storage misconfig

- Verify: bucket public read/write
- Manual: `aws s3 ls s3://bucket --no-sign-request`, browser access to bucket URL
- Burp: n/a (out of scope tool) — manual/CLI
- ZAP: n/a
- Nuclei: `-t exposures/cloud/` (S3 bucket detection templates)
- Payload: bucket name enumeration via recon (gau/subfinder patterns)

6. Evidence to Collect

- Full request/response (headers + body) proving default cred login / exposed file / missing header

- Screenshot of admin panel post-auth (redact sensitive data)
- Downloaded sensitive file listing (e.g., `.git/config` content, `.env` keys — redact values)
- Response diff: with vs without security header
- Stack trace excerpt showing framework/version/path disclosure
- Nuclei/ZAP scan report snippet with template ID/rule ID

7. Good PoC

- Step-by-step repro: URL/endpoint → request sent → response received → data/access obtained
- Show actual sensitive artifact retrieved (masked secrets) — e.g., `.git` dump → reconstructed source file
- For creds: successful authenticated session screenshot + session token (redacted)
- Business impact statement: "this yields full source code / DB creds / admin access"
- Remediation-ready: exact header/config that's missing

8. Attack Chains

- Exposed `.git` → source code → hardcoded DB creds → direct DB access
- Debug mode → stack trace → internal path/framework version → targeted exploit (known CVE)
- Default creds on admin panel → RCE via file upload/plugin (Tomcat Manager WAR deploy)
- Missing CSP → reflected XSS impact escalation
- Public S3 bucket → PII dump → regulatory breach
- Actuator heapdump/env → secrets → lateral movement to other services
- Directory listing → backup.sql → full DB dump
- Verbose error → SQL error → blind SQLi confirmation

9. False Positives

- Security headers present at CDN/WAF layer but missing at origin (or vice versa) — check both
- Default page (404/500) mistaken for debug leak — verify actual stack trace present
- "Directory listing" on intentionally public static asset folder
- Default creds rejected due to account lockout/2FA — flag as weakness not full compromise
- Nuclei generic tech-detection template flagged as vuln (informational, not exploitable)
- Version banner disclosure without corresponding known CVE — informational severity only

10. Prerequisites for Exploitability

- Misconfigured resource must be network-reachable (not just present)
- No compensating WAF/network ACL blocking the path

- Default creds not yet rotated
- Debug/verbose mode toggle left on in target environment (not just staging)
- Sensitive file must contain exploitable data (not empty/rotated secrets)

11. Attacker Achieves

- Full source code disclosure (via .git/.svn)
- Credential/secret harvesting (.env, config files, heapdump)
- Admin/privileged access (default creds)
- Internal architecture mapping (stack traces, headers)
- RCE (via exposed management consoles — Tomcat/Jenkins/Actuator)
- Mass data exposure (public cloud storage)
- Clickjacking/XSS amplification (missing headers)

12. CIA Impact Table

Variant	C	I	A	Example CVSS Vector (v3.1)
Exposed .git/source code	High	Low	None	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Default creds → admin access	High	High	High	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Debug mode/stack trace	Medium	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Public cloud storage bucket	High	Med	Low	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Missing security headers	Low	Low	None	AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N
Directory listing	Medium	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Actuator/heapdump exposure	High	Med	Low	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:L

13. Typical CVSS Severity/Vector

- Range: **Low** → **Critical** — entirely context/data-dependent (widest spread of any OWASP category)
- Info disclosure only (headers): CVSS ~3.5–5.0 (Low/Medium)
- Sensitive file/source exposure: CVSS ~7.0–8.0 (High)
- Default creds → full admin/RCE: CVSS 9.0–10.0 (Critical) — `AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H`
- Base severity always stated **per-variant**, not as single blanket score — interview point to emphasize

14. Business Impact (Financial App Context)

- Exposed `.git`/source → business logic, fraud-detection thresholds, encryption keys leaked → direct fraud enablement
- Public cloud bucket with PII/KYC docs → **PCI-DSS Req 3/7** violation, **GDPR Art. 32** breach, mandatory disclosure
- Default creds on admin/payment gateway console → direct financial fraud, unauthorized fund transfer
- Debug mode leaking DB connection strings → path to core banking DB compromise
- Regulatory: RBI cybersecurity framework (India context), PCI-DSS fines, GDPR up to 4% global turnover
- Reputational: source code leak → competitor/attacker advantage, customer trust loss
- Regulatory reporting timelines triggered (breach notification within 72h GDPR)

15. Indicators/Signals

- Response headers missing/inconsistent across endpoints
- 500 errors returning stack trace instead of generic message
- `/.git`, `/.env` returning 200 instead of 403/404
- Login succeeds with vendor default creds
- Directory index page (`Index of /`) rendered
- Verbose `Server` / `X-Powered-By` headers with exact version
- Debug query params accepted (`?debug=true`) and change response verbosity
- Admin panel reachable without auth prompt from public IP

16. Root Cause (Code/Config Level)

- Framework debug flag left true in prod build:

```
DEBUG = True    # Django settings.py deployed as-is
app.run(debug=True) # Flask
```

- Deployment script copies entire repo (`cp -r . /var/www/`) including `.git`
- No `.dockerignore` / `.gitignore`-aware build → secrets baked into image
- IaC template defaults: S3 bucket ACL `public-read` not overridden
- No centralized security header middleware — each service configures independently or not at all
- Default install left un-hardened (sample apps, manager console) — no post-install checklist enforced
- No environment-specific config separation (same config file dev/prod)

17. Remediation / Secure Design

- **Deny-by-default config baseline:** hardened base image/template, explicit opt-in for features
- CI/CD build step: strip `.git`, `.env`, dev tools from prod artifact

```
rsync -av --exclude='.git' --exclude='.env' ./src/ ./dist/
```

- Centralized security header middleware (helmet.js / Spring Security headers / Django SecurityMiddleware)
- Environment-based config injection (secrets via vault/KMS, not files)
- Automated config drift detection (compare prod config vs golden baseline)
- Disable debug/profiling endpoints via build flag, not runtime toggle
- IaC security scanning pre-deploy (Checkov, tfsec) enforcing private-by-default storage
- Mandatory credential rotation on install; disable/remove default accounts
- Segmented environments — no shared config between staging/prod

18. Compensating Controls

- WAF rule blocking access to `/.git/`, `/.env`, `/actuator/*` paths
- Network ACL/firewall restricting admin console access to internal IPs/VPN
- Reverse proxy strips verbose headers before reaching client
- Rate limiting + alerting on 403/404 spikes to sensitive paths (scan detection)
- IP allowlisting for management interfaces
- WAF/CDN-injected security headers if origin can't be changed immediately

19. Repeated Occurrence — Signals

- No config hardening baseline/checklist enforced across teams
- No CI/CD gate scanning deployment artifacts pre-release
- No centralized DevSecOps ownership of infra hardening
- Config drift not monitored — no golden-image comparison
- Indicates **immature SDLC**: security bolted on post-deploy, not designed-in
- Cross-team inconsistency → no shared IaC modules/security defaults library

20. Upstream Improvements

- **SAST/IaC scan:** Checkov, tfsec, Terrascan on IaC pre-merge
- **Secrets scanning:** gitleaks/truffleHog pre-commit hook + CI gate
- **Linter:** framework-specific (django-security linter, eslint-plugin-security)

- **CI/CD gate:** build artifact scan for `.git` / `.env` before deploy
- **Framework protections:** enforce SecurityMiddleware/helmet as mandatory shared wrapper (mirrors IDOR shared-library strategy)
- **Design checklist:** pre-deployment hardening checklist (CIS Benchmarks) as release gate
- **Golden image/config baseline** + automated drift detection (AWS Config, Azure Policy)
- **DAST scheduled scan:** Nuclei `exposures/` + `misconfiguration/` templates in nightly pipeline

21. Sample Triage Response

Confirmed — `.git/config` publicly reachable at `<endpoint>`, returns 200 w/ full repo metadata. Verified via `git-dumper` reconstruction, exposing [N] source files including [config file] with embedded [credential type — redacted]. Severity: High (CVSS 7.5, AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N). Business impact: source code + credential disclosure — recommend removing artifact from webroot and rotating any exposed secrets immediately. Awarding per [program] Security Misconfiguration classification.

22. Sample Developer Remediation Note

Issue: `.git` directory deployed to production webroot at `<path>`, publicly downloadable. **Fix:** Exclude `.git`, `.env`, `*.bak` from deployment artifact via build pipeline (`rsync --exclude` or Dockerfile `.dockerignore`). Add CI gate step scanning final artifact for VCS metadata before deploy. **Immediate action:** Remove `.git` from prod server now; rotate any credentials found in exposed history/config. **Verify fix:** `curl -I https://<host>/.git/HEAD` → expect 403/404. **Retest:** QA/security to confirm removal + CI gate active before closing ticket.

23. Real-World CVE / Incident

- **Capital One breach (2019):** misconfigured WAF + overly permissive IAM role → SSRF chained with S3 misconfig → 106M records exposed (classic misconfig+SSRF chain example)
- **CVE-2025-66036:** publicly exposed `.git` directory leading to full source code/secrets disclosure — reported via bug bounty, Hall of Fame acknowledgment
- **Spring Boot Actuator/heapdump exposure:** repeatedly found in bug bounty programs — `/actuator/heapdump`, `/actuator/env` leaking secrets/memory dumps, automatable via Nuclei
- **RabbitMQ default creds (bug bounty):** default admin console credentials → full queue/message access, high-value finding
- **Twitch (2021):** source code + payout data exposed via misconfigured internal Git server access

30-sec interview answer: "Security Misconfiguration is any insecure default or missing hardening setting across app/server/cloud layers — debug mode, default creds, exposed `.git` / `.env`, missing

security headers, open S3 buckets. It's OWASP A05:2021, CWE-16 family. Unlike other classes it's config-state driven, not payload-driven — I test via recon, header audits, content discovery (ffuf/gau), default cred checks, and Nuclei's `exposures/` and `misconfiguration/` template sets. Severity swings Low to Critical depending on what's exposed — a missing header is Low, exposed `.git` with DB creds is High-to-Critical. Root cause is almost always deployment/CI hygiene, not code logic, so remediation is upstream: strip secrets/VCS from build artifacts, enforce security headers via shared middleware, IaC scanning pre-deploy, and a hardening checklist as a release gate."